Membrane Neuron Simulator

User manual

W. van Vuuren

October 24, 2026

Abstract

The Membrane Neuron Simulator computes the static equilibrium of fluid-driven soft devices: rubber membranes between fluid chambers that push on each other, on rigid stops and on soft tubes. You draw the device in CAD, import it as STEP, give every solid a role, and solve. This manual explains how to use the program, from installing it to characterising a complete mechanical neuron.

Contents

1 Overview **3**

2 Installation and start-up **3**

3 Preparing the CAD model **4**

4 The window **5**

5 Inputs → pre-activation (the neuron) **5**
5.1 Workflow . . . 5
5.2 Roles and properties . . . 6
5.3 Solver settings . . . 7
5.4 Results . . . 7
5.5 Sweeps . . . 7

6 Pre-activation → activation (the valve) **8**
6.1 Building the model . . . 8
6.2 Flow connections . . . 8
6.3 Flow resistance equations . . . 9
6.4 Study settings and simulating . . . 9
6.5 Outputs and results . . . 9

7 Full neuron **10**
7.1 Linking a design into a neuron . . . 10
7.2 Solving and characterising . . . 10
7.3 The valve on its own . . . 11
7.4 Out-of-date components . . . 12

8 Analysis **12**

9 Networks of characterised neurons **13**

10 Design studies with an AI agent **16**

11 Troubleshooting **17**

12 Known limitations **17**

13 Files **17**

1 Overview

A mechanical neuron has two stages, and the program has one workspace (a *space*) for each, plus one for the combination, one to analyse its characterisation and one for automated studies. The spaces are the five tabs at the top of the window:

Tab	What you do there	Saved as
Inputs → pre-activation	Input chambers push on membranes into a closed pre-activation chamber. Solve and sweep the inputs.	`.mns`
Pre-activation → activation	A membrane, driven by the pressure difference Δp across it, squeezes a soft tube; gas flows through the tube. Compute the *activation function* once and save it as a design.	`.mad`
Full neuron	Link a `.mns` and a `.mad`: the activation design replaces one neuron membrane. Solve and characterise the complete neuron over many parameters.	`.mfn`
Analysis	Plot a full neuron's stored characterisation against its swept parameters (read only, Section 8).	—
Network	Use characterised neurons (and valve designs) as blocks and wire them into a network that is evaluated from the stored data (Section 9).	`.mnn`
Agentic research study	Let an AI design agent explore designs through the program's command line, and follow it live (experimental side project, Section 10).	a study folder

Each tab is a complete window with its own menus, toolbar and model, so you can have a neuron open in the first tab and a valve in the second at the same time.

Units and sign conventions. Lengths are in mm, Young's moduli in MPa, pressures in kPa and volumes in mm^3. All pressures are **gauge**: the surroundings are 0 kPa. A membrane face that touches no chamber sees 0 kPa. The only exception is the flow-resistance equations of the activation space, which use SI units (Section 6.3).

What is simulated. Everything is quasi-static: the program finds the equilibrium shape for the given pressures, not the motion towards it. Membranes are large-strain rubber sheets (incompressible neo-Hookean by default); shells add bending stiffness; fluid chambers either hold a set pressure or change pressure with their volume; rigid bodies and soft solids are kept apart by contact. The solver is a Newton–Raphson method that ramps the pressures up in load steps.

2 Installation and start-up

The program is written in Python (3.10 to 3.12; tested with 3.11) and runs on Windows. To install it:

1. install Python, from Anaconda or from python.org (tick *Add python.exe to PATH*);
2. double-click `install.bat` in the `MembraneNeuronSimulator` folder. It creates a Python environment in `%LOCALAPPDATA%\MembraneNeuronSimulator\venv` and installs the packages listed in `requirements.txt` (`numpy`, `scipy`, `torch`, `gmsh`, `PyQt5`, `pyvista`, `pyvistaqt`,

`matplotlib` and a few more). The first run downloads about 1 GB; run it again to repair the environment.

The environment is kept in a short path outside the project folder because Windows limits paths to 260 characters. Without the installer, `python -m pip install -r requirements.txt` installs the same packages into any Python 3.10–3.12; use PyQt5 only, since PySide6 or PyQt6 next to it can break the 3D view. Building PDF reports needs a LaTeX installation (MiKTeX) with `pdflatex` on the path, and the agentic research study needs Claude Code.

Start the program in one of these ways:

- double-click the desktop shortcut or `Membrane Neuron Simulator.bat` (it uses the environment of `install.bat`, otherwise Anaconda's Python);
- from a terminal in the `MembraneNeuronSimulator` folder:

```
python run_app.py                    # empty window
python run_app.py model.step         # import a STEP file
python run_app.py project.mns        # open a neuron project
python run_app.py valve.mad          # open an activation design
python run_app.py neuron.mfn         # open a full neuron
python run_app.py --analyse neuron.mfn   # ...in the Analysis tab
python run_app.py network.mnn        # open a neuron network
```

The program runs without a console. If it misbehaves, errors are written to `%USERPROFILE%\.membrane_neuron_simulator\errors.log` and `crash.log`.

3 Preparing the CAD model

The simulator does not draw geometry; it reads a STEP file in which every region is its own solid body. These rules apply to every space.

1. **One solid per region.** Each membrane or shell is a thin solid; each fluid chamber is a solid that fills the fluid space; frames, housings and stops are solids. In Fusion 360 a multi-body part or an assembly both work.
2. **Bodies touch, they do not overlap.** A membrane and the chambers on either side of it share faces. A chamber fills exactly the space between its walls. Overlaps make the coupling detection unreliable.
3. **A membrane is a plate with one main face.** The program builds the membrane's mid-surface from its *largest* CAD face, moved half the measured thickness into the solid. Make each membrane a single flat or curved face (a disc, plate or dome), not a face split into patches.
4. **A chamber must cover the membrane face it loads.** The coupling is found geometrically: points just beyond each membrane face are tested against every chamber solid.
5. **Name your bodies.** Part names (or Fusion 360 body names) are carried over and appear in the model tree. Names such as `Membrane_1`, `Chamber_A`, `Housing` or `TubeFluid` let the program guess roles for you.
6. **Export in millimetres** as STEP (`.step` or `.stp`).

Tip

Two generated examples show a correct model: `python examples/make_neuron_step.py` writes `examples/soft_neuron.step` (a neuron), and `python examples/make_activation_step.py` writes `examples/squeeze_valve.step` (a valve with `TubeFluid`, `InletFluid` and `OutletFluid`).

4 The window

The neuron and activation tabs share the same layout:

- **3D view** (centre). Left-drag rotates, right-drag or the wheel zooms, middle-drag pans. Click a part to select it; Ctrl+click adds parts to the selection. Clicking the same spot again selects the part *behind* the selected one, so you can reach inner parts without hiding anything.
- **Model tree** (left). Every solid with its role colour. Untick a part to hide it. Right-click a selection for **Assign role**, **Show only these** and **Show all**. In the activation space the tree also lists the **Flow connections**.
- **Side tabs** (right): **Part** (role and properties of the selection), extra tabs of the space, **Solver**, **Results**.
- **Message log** (bottom). Everything the program does and finds: couplings, warnings, convergence. Long jobs run in the background; the **Cancel** button next to the log stops them.
- **View toolbar**: view mode, element edges, a **Transparency** slider for unselected parts and a **Section** cut (axis plus position slider).

Key	Action	Key	Action
Ctrl+O	Open STEP	F5	Solve / Simulate
Ctrl+Shift+O	Open project / design	F6	Sweep (neuron)
Ctrl+S	Save	1 2 3	Model / Mesh / Results view
Ctrl+Shift+S	Save as	E	Show element edges
Ctrl+M	Mesh	R	Reset camera
Ctrl+K	Check model	F1	Quick guide

5 Inputs → pre-activation (the neuron)

5.1 Workflow

1. **File → Open STEP** (Ctrl+O). Every solid appears in the model tree as *Unassigned*.
2. **Assign roles** (Section 5.2). Select one or more parts and choose the role in the **Part** tab, or right-click → **Assign role**. **Edit → Auto-assign roles from names** fills in unassigned parts from their names. To reach parts inside a housing, untick the housing, click through it, or use a section cut.
3. **Set the properties** of each part in the **Part** tab. Thicknesses are measured from the CAD when the role is assigned. Selecting several parts of the same role edits them together.
4. **Simulation → Check model** (Ctrl+K). This meshes the model and writes to the log which chamber loads which membrane, from which side, and any warnings (for example a chamber that touches no membrane). Fixed nodes are drawn in blue in the mesh view.

5. **Simulation → Solve** (F5), then read the **Results** tab (Section 5.4).
6. **Simulation → Sweep** (F6) to map the response over one or two inputs (Section 5.5).
7. **File → Save project** (Ctrl+S) stores roles, properties and solver settings in a `.mns` file. The STEP file is referenced, not copied; keep it next to the project.

5.2 Roles and properties

Role	Simulated as
Membrane	Rubber sheet on the solid's mid-surface, no bending stiffness.
Shell	The same, with bending stiffness (use it for stiffer, thicker sheets).
Rigid body	Undeformable obstacle (housing, stop). Membranes cannot pass through it.
Fluid chamber	Pressure on every membrane or shell face it touches.
Activation membrane	A membrane replaced by a pre-computed activation design (`.mad`); see Section 7.
Ignore / Unassigned	Left out of the simulation.

Membrane and shell properties.

Material model Neo-Hookean (incompressible rubber, default) or St. Venant–Kirchhoff (then also *Poisson's ratio*, default 0.45).

Young's modulus Default 0.5 MPa for a membrane, 1.0 MPa for a shell. Silicone rubbers are typically 0.05–1 MPa.

Thickness Measured from the CAD solid when the role is assigned; type a value to override it.

Elements per shortest side Mesh density along the shorter in-plane side (default 10). 8–14 is the useful range; keep it the same across designs you compare.

Fixed edges *All boundary edges* (default) or only *Edges touching rigid bodies*.

Edge support (shells) *Clamped* edges keep their slope, *Pinned* edges can rotate.

Pre-tension (membranes) Isotropic in-plane tension in N/mm before any pressure is applied.

Rigid body. Only *Elements per shortest side* (the surface mesh used for contact). Curved surfaces need a finer mesh, because contact is checked at the membrane's nodes against this mesh.

Fluid chamber. The *Pressure model* decides how the pressure behaves; the chamber is coloured accordingly in the 3D view.

Model	Colour	Behaviour and settings
Constant pressure (input)	green	Held at *Pressure*. These are the neuron's inputs.
Closed: ideal gas (isothermal)	blue	Sealed air, at *Pressure* when sealed. Boyle's law on the gas share. *Ghost volume* (mm^3) adds volume that is connected but not drawn (tubing). *Incompressible fluid* (%) fills part of the chamber with liquid; the colour gets darker as it rises.
Closed: incompressible	purple	Sealed liquid, $P = P_0 - K(V - V_{\text{fluid}})$. *Stiffness* in kPa per % volume change (default 10; water is about 22 000; above about 1 000 the results barely change while solving gets slower). *Fluid volume* defaults to the body volume; less liquid than the cavity gives suction, more inflates it.
Vent (open to surroundings)	transparent	Always 0 kPa.

5.3 Solver settings

The **Solver** tab holds settings that rarely need changing:

Load steps (default 10) The pressures are ramped from 0 to full in this many steps. A step that fails is cut automatically, so this is a starting value.

Max Newton iterations (default 40) per load step.

Residual tolerance (default 10^{-8}) relative convergence criterion.

Contact stiffness (MPa/mm) The penalty stiffness of contact. 0 (default) chooses it automatically so that the penetration stays at about 1–5 % of the thinnest part at the highest pressure.

5.4 Results

After a solve, the view switches to **Results** (3). The **Results** tab offers:

- the contour field: displacement magnitude, displacement X, Y or Z, or area stretch (how much a membrane's area has grown; above about 2 the rubber is stretched far);
- a slider through the converged load steps; step *0 (start)* shows the chamber pressures on the undeformed shape (for example the initial suction of an under-filled liquid chamber);
- *Same colour range for all load steps*, to compare steps by eye;
- a table of every chamber's pressure and volume change;
- export to CSV (table) and VTK (meshes, for ParaView), and **File → Screenshot**.

5.5 Sweeps

Simulation → Sweep (F6) opens the sweep dialog.

1. Choose one or two input chambers with a range (*From*, *To* in kPa) and a number of *Points*. Two inputs give a grid.
2. Press **Run**. Each point starts from its neighbour's solution, so a sweep is several times faster than separate solves.

The table lists every chamber's pressure at every point, and the outputs of any linked activation design. The plot shows the chosen *Plot output* against the inputs (a line, or a heat map for two inputs). **Export CSV...** writes all points, with the chambers' volume changes.

Tip

To sweep more than two parameters, or other chamber values than input pressures, link the neuron in the **Full neuron** tab, run a characterisation (Section 7) and look at it in the **Analysis** tab (Section 8).

6 Pre-activation → activation (the valve)

The device is a valve: the pressure difference Δp across a membrane squeezes a soft tube, through a part bonded to the membrane (a pusher, rigid or deformable), against a rigid body. Gas flows through the tube from a supply to a sink. The program computes how the tube's cross-section, the mass flow and the pressures along the tube change with Δp. You simulate it once, save it as a design (`.mad`) and then use it as a part in a neuron without simulating it again.

6.1 Building the model

In CAD, make solids for the membrane, the pusher (touching the membrane's face and the tube), the tube, the rigid body behind it, and the gas: one body filling the tube exactly, a supply body against the tube's inlet end and a sink body against its outlet end. The tube needs two planar end faces normal to its axis.

Role	Simulated as
Membrane / Shell	As in the neuron space, clamped along its edges. Positive Δp pushes it towards the tube. It is bonded to every free rigid body or solid its face touches.
Channel (tube)	3D solid (quadratic 10-node tetrahedra, compressible neo-Hookean, $E = 0.5$ MPa, $\nu = 0.45$), fixed at its two end faces.
Fluid, Constant pressure	A supply or sink held at *Pressure*.
Fluid, Dynamic pressure	The pressure follows from the flow. The fluid filling the tube is cut into *Segments* (default 10) along it, each a flow resistance in series with its own pressure on the tube wall.
Solid	Deformable 3D part (for example a soft pusher). *Support*: free (held by what it is bonded to) or fixed where it touches fixed rigid bodies.
Rigid body	*Motion*: Fixed (an obstacle, such as the block the tube lies on) or Free (moves and tilts as a rigid body, for example a rigid pusher).

Watch out

A deformable (Solid) pusher can fail to converge on round tubes. Use a free rigid pusher there.

6.2 Flow connections

Wherever two fluid bodies touch, and wherever a dynamic fluid faces the outside, the program creates a *flow connection*. They are listed under **Flow connections** in the model tree and in the **Flow** tab; selecting one highlights its contact face. Each is one of:

- **Opening** (no resistance) — the default between two fluids;
- **Orifice** — a resistance given by your equation;
- **Closed** — the default towards the outside.

6.3 Flow resistance equations

A resistance equation gives the pressure drop in **Pa** for a mass flow `mdot` in **kg/s**, in SI units. Available variables:

`mdot`	mass flow (kg/s)	`A`, `P`	area (m^2), perimeter (m)
`rho`	density at the mean pressure	`h`, `w`	height and width of the section
`rho_up`	upstream density	`L`	segment length
`mu`	viscosity (Pa s)	`Dh`	hydraulic diameter $4A/P$
`p`, `p_up`, `p_down`	mean, up-, downstream pressure		

A segment uses the smallest cross-section in it; a connection uses its contact face (for a typical model, the whole bore). The defaults are

- segment (laminar): `32*mu*L*mdot/(rho*A*Dh**2)`;
- orifice: `(mdot/(0.61*A))**2/(2*rho_up)`.

For a smaller orifice than the contact face, set its **Orifice area A** (mm^2) in the **Flow** tab: the equation's `A` is then that area (`P`, `h` and `w` stay those of the contact face, `Dh` follows `A`); 0 (*contact face*) uses the face. The area can be swept with the valve on its own in the **Full neuron** tab (Section 7.3).

Tip

Size an orifice between the resistance of the open and of the closed tube. An orifice much smaller than the closed tube takes the whole pressure drop, and the tube then never controls the flow.

6.4 Study settings and simulating

The **Study** tab sets the Δp sweep and the gas:

Setting	Default	Meaning
Δp from / to, Points	0 / 20 kPa, 21	the pre-activation range
Pressure tolerance	0.01 kPa	structure–flow iteration converged
Max flow iterations	30	per Δp point
Gas constant R, Temperature	287.05 J/(kg K), 293.15 K	ideal gas, $\rho = p_{\text{abs}}/(RT)$
Atmospheric pressure	101.325 kPa	gauge to absolute
Viscosity μ	1.81×10^{-5} Pa s	air

Press **Simulate** (F5). At every Δp the structure and the flow are iterated until the pressures on the tube wall stop changing; the live plot follows the newest point.

6.5 Outputs and results

A dynamic fluid has a list of named *Outputs*: the gas pressure of one segment, for example *activation* at segment 10. Add, rename or remove them in the **Part** tab; choosing a segment highlights it in yellow in the 3D view. Changing outputs does not require a new simulation.

The **Results** tab shows the minimum tube area and the mass flow against Δp, the pressures at the tube ends and at the outputs against Δp, and the area and pressure profile along the tube at the selected point. **Save design...**, **Export CSV...**, **Export VTK...** and **Screenshot...** are underneath.

Saving. **File → Save design** (Ctrl+S) stores the roles, connections, settings, the results and the deformed shapes in one `.mad` file. Opening it shows the curves without simulating again. From Python, `ActivationDesign.load("valve.mad")` gives `.area(dp)`, `.mass_flow(dp)` and `.end_pressure(dp)`.

Watch out

A design is valid only for the supply and sink pressures it was simulated with. Change those and simulate again.

7 Full neuron

The full neuron combines a neuron project (`.mns`) and an activation design (`.mad`). One membrane of the neuron is replaced by the design's pre-computed curve of swept volume against Δp, so the valve is not simulated again.

7.1 Linking a design into a neuron

There are two ways:

- **In the neuron space**: give the membrane the role *Activation membrane*, choose the `.mad` file under *Activation design* and, if needed, the *Driving chamber* (the chamber whose pressure pushes the membrane towards the tube; *Automatic* takes the closed chamber it touches). Solves and sweeps then also report the design's outputs.
- **In the Full neuron tab** (recommended for characterisation):
  1. **File → Import inputs to pre-activation...** (Ctrl+I) loads a `.mns`; a copy is embedded in the full neuron, the original is not changed.
  2. **File → Import pre-activation to activation...** (Ctrl+Shift+I) loads a `.mad`. It must have been simulated with a version that records swept volume; otherwise simulate it again.
  3. In the **Link** tab, choose the neuron part to replace and the driving chamber.

Copies of both imported files are kept in the full neuron, so it keeps working if the files are moved, and later changes to them do not reach it until you update (Section 7.4).

The model tree shows two groups, *Inputs to pre-activation* and *Pre-activation to activation*. Click a group to move or rotate it in the view (display only). Only chamber values (pressures, stiffness, volumes) and the material of the simulated membranes and shells (Young's modulus, thickness, pre-tension, and Poisson's ratio for St. Venant–Kirchhoff) can be edited here, not the roles, chamber or material models, the mesh, the supports or the design. A thickness changed here changes the stiffness and the contact offset; the sheet's mid-surface stays where the CAD puts it.

7.2 Solving and characterising

Solve once at the set values (F5) solves the neuron at the current chamber values.

Run characterisation (F6) solves a grid. In the **Record & run** tab:

1. add sweep axes with **Add parameter**: a part, a field (any numeric chamber value, or a membrane's or shell's Young's modulus, thickness or pre-tension), from, to and points. You can sweep as many as you want; the number of solves is the product of the points. A new material axis starts at half to twice the current value;

2. to change the same field of several parts together (for example the thickness of both activation membranes, so the neuron stays symmetric), tick the other parts under **Also**. They take the axis' value at every point, and the axis is named `A+B.field` in the dataset;
3. tick what to record: chamber pressures `P:<chamber>` and volume changes `dV:<chamber>`, the pre-activation `dp`, the design outputs `out:<name>`, the tube area and the mass flow;
4. optionally *Store the deformed shapes*, to see every point in 3D after reopening (larger file).

The grid is solved in a snake-like order so that each point starts from a prediction made from its neighbours. Dense axes are cheap: most of the cost is in the first few kPa where membranes are slack.

When the run finishes, the dataset is stored in the `.mfn` file and the file is saved. Opening the file later shows the stored dataset; a warning appears when the model has changed since. The **Results** tab lists every point; **Export CSV...** writes the table. From Python, the stored dataset can be interpolated at any parameter values without simulating again.

Watch out

If Δp leaves the range the activation design was simulated over, the result is **extrapolated**: rows are marked orange, the 3D view shows a red overlay and a message lists the points. Simulate the design over a wider Δp range.

Analyse... in the **Results** tab (**Simulation → Analyse characterisation**, F7) saves the full neuron and opens its characterisation in the **Analysis** tab.

7.3 The valve on its own

Import only a pre-activation → activation design (`.mad`) and no neuron to characterise the valve itself. It is then *simulated again at every point* (as in its own tab, with the same mesh), so its own parameters can be changed and swept:

- `Pre-activation.dp`: the pressure difference across its membrane (its value when not swept is set by clicking the *Pre-activation to activation* group);
- the supply and sink pressures (constant fluids);
- the orifice areas: every orifice connection is listed under the design in the model tree; click it to set its area (0 = the contact face);
- the Young's modulus, thickness and pre-tension of its membrane, and the Young's modulus and Poisson's ratio of its tube and solid parts;
- the size of its membrane in its own plane (*Size along x/y/z*, the two in-plane directions are offered): the membrane's CAD body is stretched about its centre to that size and the valve is meshed and built again. The parts around it (e.g. the pusher bonded to it) keep their CAD size. A size axis' **Also** menu offers the membrane's other in-plane size: tick it to keep the membrane square (both sides take the axis' value). Size axes are solved outermost, so the valve is remeshed once per size; the stored 3D shapes keep one mesh per size, and between sizes the 3D view shows the nearest one.

Recorded: the named outputs, the tube area, the mass flow, the membrane's swept volume, the pressure at the tube's downstream end and the pusher travel. Δp is always solved as the innermost axis, each point starting from the one before (from rest when that fails), as along the sweep in the valve's own tab. Each point costs a full valve solve, so keep grids small. Changes made here are kept in the `.mfn`, on top of the embedded design; the `.mad` is not changed. The dataset opens in the **Analysis** tab and as a block in the **Network** tab like any other.

Watch out

Sweeping the supply and sink pressures across each other flips the tube's flow direction, but the segment numbering (and so the outputs) was fixed when the design was built.

7.4 Out-of-date components

As in a CAD assembly, the full neuron keeps using the versions of the `.mns` and `.mad` it imported. When you save changes to one of those files (in its own tab or anywhere else), the full neuron notices within a couple of seconds:

- its group in the model tree turns orange with a warning sign, and hovering it says why;
- **Update** appears in the toolbar (with the number of out-of-date components) and in the **File** menu;
- the status bar and the log name the changed file. A file that is no longer found is reported in red; the copy kept in the full neuron is then used.

Right-click a group for **Update from <file>** or **Open <file> in its tab**; clicking a group shows the same two buttons in its properties.

Updating a **neuron** shows what changed: values changed in the `.mns` since it was imported are taken, values you changed in the full neuron (fluid parameters, membrane and shell materials, solver settings) are kept, and for a value changed in both places a check box chooses which wins. If the linked membrane is no longer a membrane, the link is cleared. Updating a **design** takes the new version at once (it must have simulated results). Results and the stored characterisation made with the old version are then reported as outdated: run them again.

8 Analysis

The **Analysis** tab plots a full neuron's stored characterisation without simulating anything. Open a `.mfn` with **File → Open full neuron...** (Ctrl+O), or press **Analyse...** in the **Full neuron** tab. The model tree and the properties show every part's settings, read only (a swept setting shows its range and the value currently shown); change them in their own tabs.

1. Choose the result to plot as Z (anything recorded, for example the pre-activation chamber's pressure).
2. Tick swept parameters as plot axes, in order. The first gives a curve, the second a surface, the third a row of surfaces, the fourth a grid of surfaces and the fifth layers of grids: move the **scroll wheel** over the plots to go up and down the layers.
3. From the third ticked parameter on, the parameter shows *how many* of its values to use instead of a slider. Every unticked parameter is held at its slider's value.
4. *Interpolate between the simulated points*: off, only simulated points are used and the sliders snap to them; on, values in between are interpolated linearly from the neighbouring points.

All panels share one Z scale and colour bar. Red points were extrapolated beyond the activation design's range, black crosses did not converge, and gaps were not solved. *Surfaces as* switches between 3D surfaces and flat colour maps. Surfaces are drawn by the graphics card: drag to turn them (all panels turn together), right-drag or the scroll wheel to zoom (the wheel changes layers instead when five axes are ticked). Click a point in a plot to show the neuron and the design deformed there in the **3D view** (interpolated when interpolation is on); the point is ringed in orange and its values are listed under *Point shown in 3D*.

Tip

The program can also be started straight into this tab: `python run_app.py -analyse neuron.mfn`.

9 Networks of characterised neurons

The **Network** tab turns characterised neurons into blocks, in the style of Simulink, and wires them into a network. Nothing is simulated: every block looks its response up in the stored characterisation (multilinear interpolation, as in the **Analysis** tab), so a network evaluates in milliseconds and a sweep of thousands of points takes seconds. Networks are saved as `.mnn`; each component is copied into the file when it is added, so the network still opens when its sources have moved.

Building a network

1. **Add neuron...** (Ctrl+I): a full neuron (`.mfn`) with a stored characterisation (Section 7). **Add valve...** (Ctrl+Shift+I): a simulated pre-activation-to-activation design (`.mad`). They appear in the **Library**. A file dropped on the canvas is added and placed at once.
2. Drag a library entry onto the canvas, or double-click it. Besides the components, the library has the built-in blocks of Table 1 and the network's subsystems.
3. Drag from an output port (right side of a block) to an input port (left side) to wire them. An input takes one wire; a new one replaces it. An output can feed any number of inputs. Select blocks or wires and press Delete to remove them.
4. Select a block to edit it on the **Block** tab.

The canvas zooms with the scroll wheel, pans with the middle mouse button, and F fits the network to the window.

Neuron and valve blocks: inputs, parameters and constants

Every parameter that was swept in the characterisation is a *variable* of the block, and each variable is one of three kinds (the *Use as* column):

Input a port on the block, wired to another block. While nothing is wired to it, it takes its *Value*.

Parameter held at its *Value*, which training may change (training is to come), always within the characterised range. Parameters are marked with ♦ on the block.

Constant held at its *Value* and never trained.

Swept pressures start as inputs, everything else (thicknesses, moduli, pre-tensions, fill volumes, ...) as constants at the middle simulated value. The same component can be used with different values (e.g. different limiter fills) in different blocks. The *Output ports* list chooses which recorded quantities the block offers. A valve block has one input, the pre-activation Δp across its membrane, and offers its named outputs, tube area, mass flow and swept volume.

The number settings of math and activation-function blocks (a gain, a bias, a sigmoid's slope and centre, ...) are constants by default; the box next to each one makes it a parameter as well. The **Run** tab lists every trainable parameter, with where it is stored and its range, and its values can be changed there.

Looking at a block in the Analysis tab

Any block except sources, probes and ports can be plotted in the **Analysis** tab: a characterised neuron, a valve, a Sigmoid or another built-in block, or a subsystem such as a neuron followed by a sigmoid.

Analyse the response... (right-click the block, or the button on its **Block** tab; in the library, right-click a component, a built-in block or a subsystem). A dialog lists the block's input ports and the trainable parameters inside it. Tick what to sweep and set its range and number of points. Ranges start at the characterised range of the neuron variable a port drives (followed into subsystems), or at a function block's typical range; unticked inputs are held at their present value in the network. The block is evaluated on its own on every combination (looked up, nothing is simulated; thousands of points take about a second) and shown in the **Analysis** tab like a characterisation: the first two inputs are the plot axes, the others start at their present values. Red points mean that a neuron inside is outside its characterised range. There is no 3D view. The network is not changed.

Open its characterisation in Analysis... (neuron blocks and library neurons) opens the neuron's own `.mfn` with its 3D view, set up as the block uses it: its first two input ports are the plot axes, its parameters and constants are held at the block's values (interpolation is switched on when a value lies between simulated points), and Z is its first output port.

A valve block's **Open the design...** opens it in the **Pre-activation → activation** tab.

Outside the characterised range a block either *extrapolates linearly* (the default for neurons) or *clamps* to the range (the default for valves, which matches how a design is used in a full neuron). Either way its outline turns red and the status bar says so. The block subtitle lists the parameters it is held at.

Watch out

A wire is an ideal signal: the receiving block does not load the sending one. That holds for a valve's outputs (a supplied gas pressure; an input chamber draws no flow at steady state). A sealed chamber's pressure, such as `P:Pre-Activation`, wired into another neuron's input chamber would in reality exchange volume with it; such wires are drawn dashed. Likewise a neuron without a linked design followed by a valve block leaves out the valve membrane's compliance in the pre-activation chamber (it adds to W_0): characterise the linked full neuron when that matters.

Block library

Table 1 lists the built-in blocks. Select a block to set its parameters on the **Block** tab. A one-input nonlinear block draws its curve on the canvas. Logic blocks take a value above their threshold (default 0.5) as true and give 1 for true and 0 for false. Invalid operations (the log of a negative number, division by zero) give NaN, which propagates to the outputs. A *Function* block also accepts numpy functions, `pi`, `e` and `sigmoid(x)` in its expression.

Subsystems: blocks built from blocks

A subsystem is a block made of other blocks, for example a characterised neuron followed by a Sigmoid as one "full neuron" block.

1. Select the blocks (drag a box around them) and choose **Create subsystem from selection** (Ctrl+G). The selection is replaced by one block. Every outside signal that fed the selection becomes an *In port*, and every signal the selection sent out becomes an *Out port*; the wires are kept.

Group	Blocks
Sources and probes	Input (network input), Constant, Output (probe)
Math	Sum (signs), Gain, Bias, Product (`*` and `/` per input), Abs, Sign, Square root, Power, Exp, Log, Min, Max, Function (an expression in `u1`, `u2`, ..., e.g. `20*sigmoid(0.3*(u1-5))`)
Activation functions	Sigmoid $\ell + (h-\ell)/(1+e^{-k(u-x_0)})$, Tanh $A\tanh(k(u-x_0))$, ReLU, Saturation, Dead zone, Step
Logic and comparison	Compare ($u_1 \lessgtr u_2$), Compare to constant, AND, OR, XOR, NAND, NOR, NOT, Switch (passes u_1 or u_3 depending on u_2)
Lookup	Lookup table (breakpoints and values, linear between them, held at the ends)
Subsystems	New empty subsystem, the network's subsystems; inside a subsystem also In port and Out port

Table 1: Built-in blocks of the Network tab.

2. Double-click a subsystem (or right-click it in the library → **Open**) to edit its contents. The location bar above the canvas shows where you are; Backspace, **Up** or a click on a name in the bar goes back. Inside, add *In port* and *Out port* blocks from the library to add ports. A port's number sets its position on the block.
3. Every block of the same subsystem uses the same contents, so editing one edits all, like a Simulink library link. **Make independent** on the **Block** tab gives one block its own copy. Inputs that should differ between copies, such as limiter fills, are best made In ports.
4. Right-click a subsystem in the library to rename, duplicate, delete or **Export** it as a library block (`.mnb`), together with the components and nested subsystems it uses. **Import block...** (or dropping the file on the canvas) adds it to another network.

Subsystems can be nested, but a subsystem cannot contain itself. Signals, values on ports and feedback loops work across subsystem boundaries: the network is evaluated as if every subsystem were expanded in place.

Running

On the **Run** tab, set the Input blocks' values. With *Evaluate on every change* ticked (the default), the network is evaluated at once and every output port shows its value on the canvas; Output blocks show theirs inside. **Evaluate** (F5) does the same on request and writes the values to the messages. Input and Output blocks inside subsystems are listed by their path (`H1/p1`). An Input block inside a subsystem that is used more than once has one value, shared by all uses (only a sweep sets one path apart): make inputs that should differ In ports.

Feedback (a wire that leads back to an earlier block, or to the block's own input) makes an algebraic loop. The outputs on the loop are solved for together, and the status bar reports whether that converged. A bistable loop has more than one solution: a sweep starts every point from the previous one, so it follows one branch, and sweeping back can show hysteresis.

Run sweep (F6) sweeps one Input block (x) over a range, optionally a second (y), and plots every Output block. If y has up to eight values each one gets its own curve; with more, every Output is drawn as a colour map. Red points were outside a block's characterised range and black crosses are loops that did not converge. **Export CSV...** writes the sweep, one row per point.

Out-of-date components

When a component's source file is saved with a different characterisation (or a changed design), the library shows it in orange as *out of date* (red if the file is missing). Right-click it and

choose **Update from source** to take the new dataset. Blocks keep their port settings where the parameters and recorded quantities still exist, and wires to ports that disappeared are removed. **Open ... in its tab** shows the source in the **Full neuron** or **Pre-activation → activation** tab.

10 Design studies with an AI agent

The **Agentic research study** tab runs a design agent (Claude Code) on a study folder and lets you follow it. The agent cannot write or run code: it only uses the program's `mns` command line, builds designs from parametric design files, runs them, and writes a LaTeX report per design and one for the study.

Watch out

The agentic research study is very experimental and a side project next to the simulator itself. Expect rough edges, and check the agent's designs and conclusions yourself before relying on them.

1. **New study...**: choose an empty folder, write the *brief* (the goal, the free parameters and their ranges, what to measure, the time budget) and add your starting models (`.mns`, `.mad`, `.mfn`). They are imported as the first designs.
2. **Start agent** opens Windows Terminal in the study folder with Claude Code running. The first time, accept Claude Code's question whether to trust the folder.
3. Watch the tab. With *Follow the agent* ticked it shows the design the agent works on, the deforming shape during solves, the agent's current command and its reason, the design list, the event log, and every plot and picture. **Open in its space** opens a design in the matching tab.
4. **Edit brief**, **Open folder** and **Study report** give quick access to the folder.

At the end, `report/study.pdf` holds the overall findings. Each design has a folder `designs/<ID>_<name>/` with `design.yaml` (parameters, geometry, roles), `model.step`, the simulator project, `results/`, `renders/` and `report/design.pdf`.

You can use the same command line yourself, in the study folder: `mns help` lists the commands, and `MANUAL.md` in the study folder documents them and the design-file format. For example:

```
mns design derive N001 thin --set Membrane1.thickness=0.3 --why "thinner"
mns sweep N002 --input Input1=0:20:5 --input Input2=0:20:5
mns report study --build
```

11 Troubleshooting

Symptom	What to try
A chamber "touches no membrane" in Check model	The chamber's faces must meet the membrane face exactly. Inspect with a section cut; fix overlaps or gaps in CAD.
A membrane's mid-surface looks wrong	The membrane must have one main face (Section 3).
The solve does not reach full load	More *Load steps*; fewer elements per side; check for a physical limit point (a membrane ballooning past its peak pressure, or snap-through). The log reports the load factor reached.
Very large area stretch (> 2)	The membrane hits a wall or is over-stretched: bigger chamber, thicker or stiffer membrane.
A full neuron component is marked out of date	Its file was saved with changes: right-click the group → **Update** (Section 7.4).
The tube never controls the flow	The orifice is much smaller than the closed tube; enlarge it.
Some Δp points did not converge	They are marked in the plot. Raise *Max flow iterations*, use a coarser tube mesh, or use a rigid pusher.
"EXTRAPOLATING"	Simulate the activation design over a wider Δp range.
The program closes or hangs	Read `errors.log` and `crash.log` in `%USERPROFILE%\.membrane_neuron_simulator`.

12 Known limitations

- Static only: no dynamics, no time.
- Contact is checked at mesh nodes; use a mesh fine enough to follow the obstacle's curvature. A sheet does not contact itself.
- A membrane's mid-surface comes from its single largest CAD face.
- Flow is steady, isothermal and lumped (uniform pressure per segment). No choked flow unless you write it into the resistance equation. A fully closed tube keeps about 2 % of its open area (the contact gap).
- A deformable Solid pusher may not converge on round tubes.
- An activation design is valid only for the supply and sink pressures it was simulated with.

13 Files

File	Contents
`.step`	Your CAD model (not changed by the program).
`.mns`	Neuron project: STEP path, roles, properties, solver settings, sweep settings.
`.mad`	Activation design: roles, connections, study settings, results and deformed shapes.
`.mfn`	Full neuron: embedded copies of the neuron and the design (with their file paths, to detect changes), the link and the stored characterisation.
`<name>.csv`	Sweep export.